



Bloque Fundamentos: Git Actions, Docker y SQL

GitHub Actions

Ignacio Reyes Vázquez, Javier Plaza Rosique

Ignacio Reyes Vázquez



Cloud Data Architect

BBVA Technology

Máster en Project Management (UNIR) 2024-2025

Máster en Data Analytics para la Empresa (EDEM) 2023-2024

Administración y Dirección de Empresas, especializado en Business Intelligence (UPV) 2019-2023

Data Scientist (Beca de Colaboración UPV) 2022-2023

Cloud Data Scientist/Data Engineer (NTT DATA) 2023-2024

Cloud Data Architect (BBVA TECHNOLOGY) 2024-Act.

LinkedIn: www.linkedin.com/in/ignacio-reyes-vazquez

Javier Plaza Rosique



Data Engineer

Cecotec Innovaciones S.L.

Data Engineer en Cecotec

Master en Big Data & Cloud en EDEM
Grado en Economía UMU

LinkedIn: www.linkedin.com/in/javiplaza

Roadmap · Fundamentos

Bloque [Fundamentos](#) del [máster](#) · del [28 sept](#) al [12 nov](#)

Parte 1 · Instalación, control de versiones y Python (28 sept – 21 oct)

FUNDAMENTOS

28–30 sept

Introducción, Instalación de Software, Linux y Git

FUNDAMENTOS

1–8 oct

Git, Programación básica con Python y Docker

FUNDAMENTOS

13–21 oct

Git Actions, Docker y SQL

Fase 01

Fase 02

Fase 03

Parte 2 · Data Science, APIs e IA generativa (26 oct – 12 nov)

FUNDAMENTOS

26 oct–2 nov

Docker y Librerías Data Science Python

FUNDAMENTOS

4–5 nov

API MGMT

FUNDAMENTOS

9–12 nov

Vibe Coding y E2E | Prueba

Fase 04

Fase 05

Fase 06

➔ Hilo conductor del alumno: [instalación](#) → [Git](#) → [Python](#) → [Docker](#) → [SQL](#) → [Data Science](#) → [APIs](#) → [Vibe Coding](#)

Objetivos de la sesión

- 01 Conocer que son las Git/Git hub Actions
- 02 Diferenciar los distintos componentes de las GithubActi ons
- 03 Crear los primeros workflows
- 04 Advertencias y Recomendaciones
- 05 Diseñar e implementar un pipel i ne de CI/CD básico para vuestro proyecto

EDEM

SECCIÓN 00

¿Qué es GitHub Actions?

Github Actions

GitHub Actions es una plataforma de integración continua y entrega continua (CI/CD), la cual, permite la automatización de tu compilación, prueba, y despliegue.

CI = Integración Continua: Cada vez que cambias algo en tu código, se revisa automáticamente.

CD = Despliegue Continuo: Si todo está bien, el programa se publica o se entrega automáticamente.

SECCIÓN 01

¿Qué pasa desde un cambio de código hasta llegar a producción?

¿Qué pasa desde que cambio código hasta que llega a producción?



Diseña tu propio pipeline

Imaginad que vuestra aplicación tiene miles de usuarios:

1. ¿Que debería ocurrir cuando alguien modifica el Código?
2. ¿Que deberíamos comprobar antes de aceptar el cambio?
3. ¿Cómo sabemos que la app sigue funcionando?
4. ¿Queremos probar el cambio antes de publicarlo?
5. ¿Cuándo debería de llegar a producción?



Una posible solución (hay miles..)



De la idea a Github Actions

Nuestro Pipeline en Github Actions:

PR → Event/Trigger

Build → Job

Test → Job

Preview → Job

Production → Job

Maquina donde "runnea" → Runner

Tareas de cada fase → Steps

```
! original.yaml
1  name: build_app
2  env:
3    myAppName: Gold
4  on:
5    push:
6      branches:
7        - main
8  jobs:
9    build-apk:
10     runs-on: macos-latest
11     steps:
12       - name: checkout code
13         uses: actions/checkout@v2
14       - name: Setup java
15         uses: actions/setup-java@v1
16         with:
17           java-version: 12.x
```

Un workflow no es mas que nuestro diagrama convertido en
YAML

Mission: Automatizar vuestra película

Mission: automatizar Su Película 🎬

Durante la sesión iremos construyendo este pipeline paso a paso.



Archivo objetivo

Aquí implementaremos el pipeline.

```
su-pelicula/  
├── .github/  
│   └── workflows/  
│       └── ci-cd.yml
```



Bonus challenges

¿Te atreves con un poco más?



Hacer que un test falle y ver qué ocurre

Comprueba cómo se detiene el pipeline.



Utilizar un Secret

Para credenciales de despliegue u otros datos sensibles.



Generar un Artifact

Guardar el build para usarlo en otra etapa.



Añadir aprobación antes de Production

Usando environments y approvals.

Pipeline final

Objetivo al final de la sesión



Mission: Automatizar vuestra película II

```
name: CI/CD - Mi Pelí
on:
  # Cuando abrimos o actualizamos una Pull Request hacia main
  pull_request:
    branches: [main]
  # Cuando hacemos merge y el código llega a main
  push:
    branches: [main]
jobs:
  # =====
  # 1. BUILD
  # =====
  build:
    name: Build
    runs-on: ubuntu-latest
    steps:
      - name: Descargar código
        uses: actions/checkout@v4
      - name: Build de la aplicación
        run: |
          echo "Construyendo Su Película..."
          echo "Build completado"
  # =====
  # 2. TEST
  # =====
  test:
    name: Test
    runs-on: ubuntu-latest
    # No ejecuto hasta que Build termine correctamente
    needs: build
    steps:
      - name: Descargar código
        uses: actions/checkout@v4
      - name: Ejecutar tests
        run: |
          echo "Ejecutando tests..."
          echo "Todas las tests han pasado"
```

```
# =====
# 3. DEPLOY PREVIEW
# Solo Pull Requests
# =====
deploy-preview:
  name: Deploy Preview
  runs-on: ubuntu-latest
  needs: test
  # Solo se ejecuta si estamos en una Pull Request
  if: github.event_name == 'pull_request'
  environment:
    name: preview
  steps:
    - name: Descargar código
      uses: actions/checkout@v4
    - name: Deploy Preview
      run: |
        echo "Desplegando entorno Preview..."
        echo "Preview desplegada"
  # =====
  # 4. DEPLOY PRODUCTION
  # Solo cuando llegamos a main
  # =====
deploy-production:
  name: Deploy Production
  runs-on: ubuntu-latest
  needs: test
  # Solo se ejecuta en un push a main
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  environment:
    name: production
  steps:
    - name: Descargar código
      uses: actions/checkout@v4
    - name: Deploy Production
      run: |
        echo "Desplegando Su Película en PRODUCCION..."
        echo "Aplicación desplegada"
```

[mi-peli.yaml](#)

EDEM

SECCIÓN 02

Componentes de las Github Actions

Componentes de las Github Actions



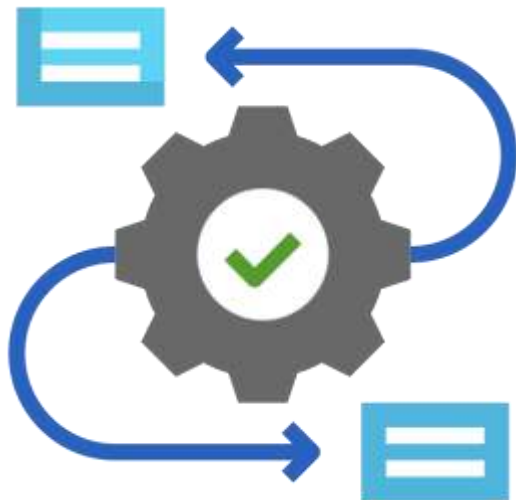
Componentes

1. Workflows
2. Events
3. Runners
4. Jobs
5. Steps

Definición

Se configura un **workflow** de GitHubActions para que active cuando se produce un **evento** que, a continuación, continúa, ejecuta una serie de **jobs** en un **runner** incluyen uno o varios **steps** que llaman a uno o varios **scripts** o una o varias **acciones**.

Componentes: Workflow



Workflow

- + Un proceso automatizado configurable que ejecutará uno o más trabajos
- + Definido en el directorio del repositorio:
`.github/workflows`
- + Un repositorio puede tener múltiples cada uno de los cuales puede realizar un conjunto diferente de tareas.

Componentes: Event



Event

- + Actividad específica que se produce en el repositorio y que puede utilizarse como trigger workflow
- + A veces se denomina «workflow triggers».
- + Eventos que desencadenan workflows

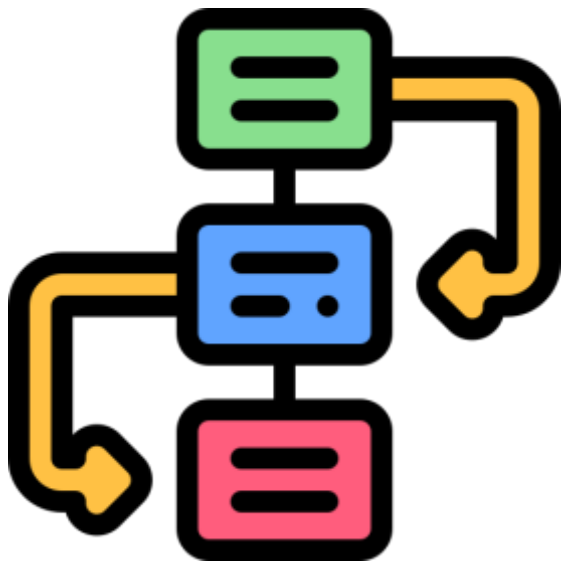
Componentes: Runner



Runner

- + Una instancia de servidor virtual que ejecuta los trabajos en tu flujo de trabajo activa
- + Proporcionado por GitHub y autohosteado
- + Windows, Ubuntu y MacOS

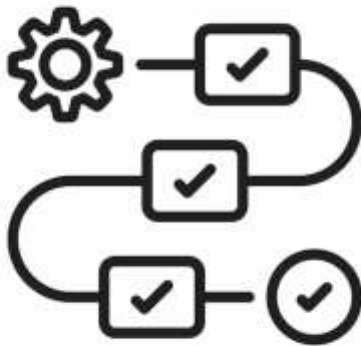
Componentes: Jobs



Job

- + Serie de steps en un workflow que se ejecutan en el mismo ejecutor.
- + Cada job se ejecuta en un entorno de ejecutor especificado por su propiedad **RUNS-ON**.
- + Los jobs se ejecutan en paralelo o de forma predeterminada.
- + Un workflow puede ejecutar un número ilimitado* de jobs.

Componentes: Steps



Steps

- + ~~X~~ sea un script de shell que se ejecutará o una acción que se llevará a cabo.
- + Se ejecutan en orden y dependen unos de otros (son una serie de tareas).
- + ~~T~~odos los pasos de un trabajo se ejecutan en el mismo ejecutor.
- + ~~T~~enen acceso al sistema de archivos y al workspace

Componentes: Actions



Action

- + Un script o pieza de automatización autónomo, autónomo, modular y compartible que realiza tarea específica.
- + Se puede encadenar en un flujo de trabajo realizar tareas de automatización complejas.

Componentes: Resumen

Piensa en GitHub Actions como una cadena de trabajo automática:

1. Evento (Event)

Algo ocurre en el repositorio. Ejemplo: subes código o pulsas un botón.

2. Workflow

Es el proceso automático que se activa cuando ocurre ese evento.

3. Jobs

Son las tareas grandes dentro del workflow.
Cada job hace una parte del trabajo.

4. Runner

Es la máquina/ordenador donde se ejecuta cada job.

5. Steps

Son los pasos pequeños dentro de un job.

6. Action

Es una tarea ya preparada que puedes usar en step para no tener que programarlo todo desde cero.



Primer Workflow

Debe contener como mínimo:

Event

Job

Runner

Step

```
First_action.yaml
```

```
on:
```

```
  workflow_dispatch:
```

```
jobs:
```

```
  say_hello:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - run: echo "Hello there!"
```

Primer Workflow

Debe contener como mínimo:

Event
Job
Runner
Step

Explicación

```
First_action.yaml
```

```
on:
```

```
  workflow_dispatch:
```

El workflow se ejecuta **manualmente** al pulsar Run workflow.

```
jobs:
```

Define los trabajos que va a realizar el workflow.

```
  say_hello:
```

Es el nombre del job

El job se ejecuta en una **máquina virtual con Ubuntu** (runner de GitHub).

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - run: echo "Hello there!"
```

Son los pasos que se ejecutan dentro del job, en orden.

Ejecuta un comando que muestra un mensaje por pantalla.

EDEM

SECCIÓN 03

Inputs

Inputs



Input

- + Solo aplicable cuando usamos el evento de `workflow_dispatch` o el de `workflow_call` en un `workflow...` o en una acción.
- + Define las entradas que el workflow/acción pueden aceptar.
- + Cada input debe ser único, alfanumérico, -, o _

Propiedades de los Inputs



Input

+ inputs. <input_id>. description lo que se le mostrará al usuario

+ inputs. <input_id>. required si es o no es obligatoria.
Opcional

+ inputs. <input_id>. default – valor por defecto a usar. Opcional

+ inputs. <input_id>. type – Tipo de dato del input.
Opcional

second.yaml

name: Welcome to the party (second)

on:

workflow_dispatch:

jobs:

say_hello:

name: "Let's greet the user!"

runs-on: ubuntu-latest

steps:

- run: echo "Hello there!"
- uses: actions/checkout@v3
- run: cat ./github/workflows/list.txt
name: display contents of list.txt
- run: echo "I am step 4!"

second.yaml

Explicación

```
name: Welcome to the party (second)
on:
```

Es el **nombre del workflow**.
Es solo para identificarlo en la
interfaz de GitHub.

```
  workflow_dispatch:
```

El workflow se ejecuta **manualmente** desde GitHub.

```
jobs:
```

```
  say_hello: Identificador del job
```

```
    name: "Let's greet the user!"
```

Es el **nombre visible del job** en la
interfaz de GitHub.

```
    runs-on: ubuntu-latest
```

Runner de Github de Ubuntu

```
    steps:
```

- run: echo "Hello there!"
- uses: actions/checkout@v3
- run: cat ./github/workflows/list.txt
- run: echo "I am step 4!"

Muestra mensaje "Hello there"

Muestra el
contenido de ese
archivo en la
consola. (Tiene
nombre descriptivo
el step)

Usa una **action oficial de GitHub**
que: Descarga el
código del
repositorio al
runner. Esto es
necesario si
quieres: leer
archivos, compilar y
ejecutar pruebas

third.yaml

```
name: Welcome to the party (third)
on:
  workflow_dispatch:
    inputs:
      thename:
        description: 'What is your name?'
        required: true
jobs:
  say_hello:
    name: "Let's greet the user!"
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello there, ${ inputs.thename }!"
```

third.yaml Explicación

```
name: Welcome to the party (third)
```

```
on:
```

```
  workflow_dispatch:
```

```
    inputs: Define los parámetros que el usuario puede ingresar al ejecutar el workflow.
```

```
      thename: Identificador del input
```

```
        description: 'What is your name?'
```

```
        required: true
```

```
jobs:
```

```
  say_hello:
```

```
    name: "Let's greet the user!"
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - run: echo "Hello there, ${ inputs.thename }!"
```

1. **description:** 'What is your name?' → le indica al usuario qué ingresar en ese input.
2. **required:** true → obliga al usuario a escribir algo; no se puede dejar vacío.

Este step ejecuta un comando de shell (echo).
\${ inputs.thename } es la **variable que contiene el nombre ingresado por el usuario** en el workflow_dispatch.

EDEM

SECCIÓN 04

Contexts

Contexts



Context

- + Proporciona acceso a información sobre ejecuciones de workflows, variables, runners, jobs y steps.
- + Expuesto como un objeto que contiene propiedades, que pueden ser cadenas u otros objetos.
- + Doce tipos de [contexts](#).
- + Los tipos de contextos disponibles dependerán del contexto en el que se acceda a ellos.

Contexts

La **sintaxis** para acceder a un contexto dentro de **workflows** y acciones compuestas:

`${<contexto>.<propiedad> }`

Ejemplo:

`${ steps.get-target-url.outputs.target_url }`  `id: get-target-url`
`run: echo "target_url=https://example.com" >>`
`$GITHUB_OUTPUT`

- run: echo "La URL es \${ steps.get-target-url.outputs.target_url }"

EDEM

SECCIÓN 05

Gestión

Gestión



Configuración Dinámica

+ Variables de Entorno (env): Permiten definir valores reutilizables en todo el workflow (ej. Nombre BDD)

```
GLOBAL_USER: "estudiante_edem"
```

```
run: echo "El usuario en Job 1 es $GLOBAL_USER"
```

+ Secretos (secrets) Datos sensibles que nunca deben de estar en el código (contraseñas, keys...) Github los enmascara directamente con ***

```
run: login -pass ${secrets.DB_PASSWORD}
```

Gestión



Matrix Strategy

- + Ejecuta un mismo Job múltiples veces con diferentes configuraciones de forma simultánea.
- + Muy utilizado para probar compatibilidad de código entre versiones (Python o SO)
- +Automatiza la redundancia y ahorra cientos de líneas de código de configuración en el archivo XML

```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest, macos-latest] # Definición de los
SO
runs-on: ${matrix.os}                                # Uso de la variable para elegir la
máquina
```

EDEM

SECCIÓN 06

Outputs, Artifacts y Custom Actions

Outputs



Output

+ Datos que devolverá una acción o paso al completarse

+ Las acciones/steps que se ejecutan más adelante en un workflow pueden utilizar el conjunto de datos de salida (output) establecido en la acción/step ejecutado anteriormente.



```
- id: get-target-url  
  run: echo "target_url=https://example.com" >>  
      $GITHUB_OUTPUT
```

```
- run: echo "La URL es {{{ steps.get-target-url.outputs.target_url  
      }}}"
```

Establecer un Output

Establecer un output:

```
echo "<output name>=<value>" >> $GITHUB_OUTPUT
```

o

```
const core = require('@actions/core'); core.setOutput("<output name>", <value>);
```

Usar ese output:

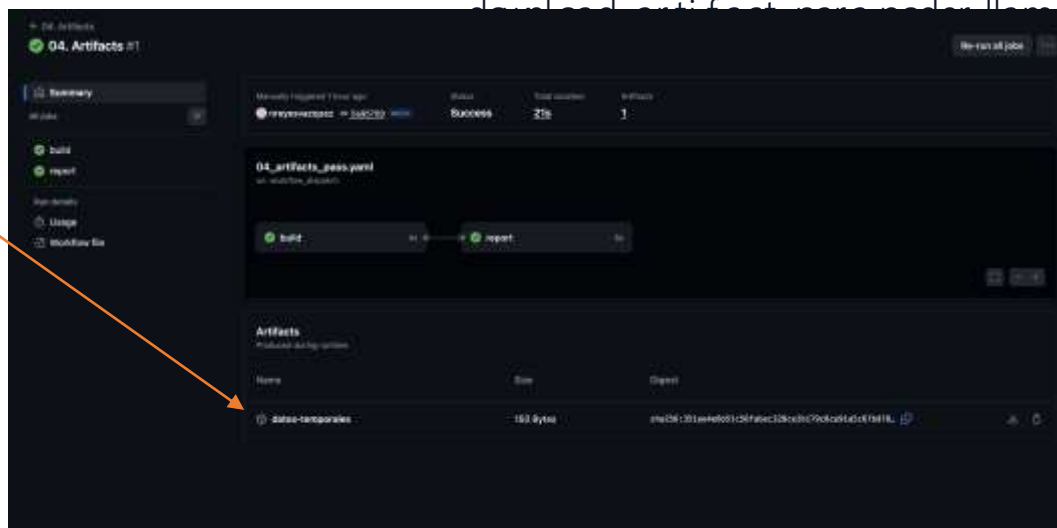
```
${ steps.<step-id>.outputs.<output-name> }
```



Artifact

+ En este caso, son archivos completos (reportes .txt, binarios, logs...) que se suben al almacenamiento de Github para ser usados por otros Jobs o descargados por el usuario

- + Se utiliza una Action como upload-artifact / download-artifact para guardarlos.



Outputs

```
outputs.yaml
```

```
jobs:
```

```
  say_hello:
```

```
    name: "Let's greet the user!"
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - id: get-time
```

```
        name: Grab the current time
```

```
        run: |
```

```
          curTime=$(date)
```

```
          echo "currenttime=${curTime}" >> $GITHUB_OUTPUT
```

```
      - run: echo "The time from the early step is  
        ${ steps.get-time.outputs.currenttime }"
```

Outputs

outputs.yaml

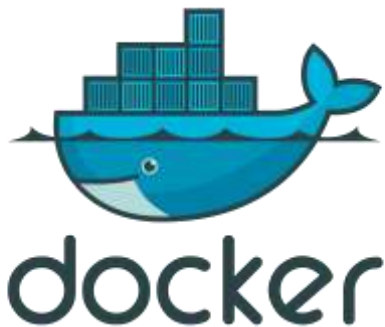
Explicación

```
jobs:
  say_hello:
    name: "Let's greet the user!"
    runs-on: ubuntu-latest
    steps:
      - id: get-time
        name: Grab the current time
        run: |
          curTime=$(date)
          echo "currenttime=${curTime}" >> $GITHUB_OUTPUT
      - run: echo "The time from the early step is
        ${ steps.get-time.outputs.currenttime }"
```

Te da la fecha y hora actual y la guardas como curTime, y luego la exportas como **currenttime**

Entras en el contexto de steps, accedes al step get-time, luego a los outputs generados en ese step y específicamente al **output** llamado currenttime, y finalmente lo muestras por pantalla con echo en el siguiente step.

Custom Actions



Custom Actions

+Tres tipos de acciones personalizadas:

Docker: ejecuta código en un contenedor.

JavaScript: usa Node.js y @actions/core.

Composite: combina múltiples pasos en una sola acción.

+Almacenamiento:

Guardadas en un repositorio.

Ubicadas en un directorio propio dentro del repo.

Reutilizables en varios workflows.

VALORACIONES

Valora la sesión para que mejoremos las próximas



Ignacio Reyes Vázquez, Javier Plaza Rosique

Git Actions

Grupo A

EDEM

¡Gracias!

reyesvazqueznacho@gmail.com

MÁSTER EN DISEÑO IA, BIG DATA Y CLOUD